

INDUSTRIAL ENERGY MANAGEMENT SYSTEM USING IIoT

*Real-time Monitoring and Control of AC Load
using RBA5D2X, MQTT and ThingsBoard*

A PROJECT REPORT

submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN ELECTRICAL AND ELECTRONICS ENGINEERING

Submitted by

Aditya Raghu

Madhav Saiprasad

Department of Electrical and Electronics Engineering

DATE OF SUBMISSION

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid growth of industrial automation and the Industrial Internet of Things (IIoT) has significantly transformed the way industries monitor, control, and optimize their operations. Among the various resources consumed in industrial environments, electrical energy is one of the most critical. Continuous monitoring of energy consumption enables industries to reduce operational costs, improve equipment utilization, detect abnormal power usage, and enhance overall system efficiency.

Traditional energy monitoring systems often require manual inspection or expensive supervisory systems, making them unsuitable for small- and medium-scale industries. The emergence of embedded systems and IoT technologies has enabled the development of compact, cost-effective, and intelligent energy monitoring solutions capable of providing real-time information through cloud-based platforms.

This internship project focuses on the design and implementation of an **Industrial Energy Management System** using the **RBA5D2X Rugged Board**, **ACS712-20A Hall-effect current sensor**, **RS485 relay module**, **MQTT communication protocol**, and **ThingsBoard Community Edition**. The developed system continuously measures the current consumed by an electrical load, calculates the corresponding power and cumulative energy consumption, and transmits the collected data to a cloud dashboard for real-time visualization and monitoring.

The system also enables remote control of the connected electrical load through the ThingsBoard dashboard. Users can switch the relay remotely and reset the accumulated energy value whenever required. The integration of embedded hardware, MQTT communication, cloud visualization, and persistent energy storage demonstrates a practical implementation of an Industrial IoT-based energy monitoring system.

The project was developed during a one-month internship at **PHYTEC Embedded Pvt. Ltd., Bengaluru**, where emphasis was placed on applying embedded Linux, industrial communication protocols, and cloud-based IoT technologies to solve real-world industrial monitoring problems.

1.2 Background

Industrial facilities consume a significant amount of electrical energy in operating machinery, lighting systems, HVAC equipment, production lines, and auxiliary services. Monitoring energy consumption at regular intervals allows industries to identify inefficient equipment, reduce unnecessary power consumption, and implement preventive maintenance strategies.

With the advancement of embedded computing platforms, real-time monitoring has become increasingly accessible. Modern embedded systems equipped with Analog-to-Digital Converters (ADC), communication interfaces, and networking capabilities can acquire sensor data, perform local processing, and communicate with cloud platforms in real time.

Industrial IoT extends these capabilities by connecting embedded devices to cloud services through lightweight communication protocols such as MQTT. This enables centralized monitoring, remote control, historical data storage, and intelligent decision-making based on real-time measurements.

In this project, the **RBA5D2X Rugged Board** serves as the embedded processing platform. The analog output from the **ACS712-20A current sensor** is sampled through the ADC of the RuggedBoard. The embedded application processes the acquired data to determine RMS current, electrical power, and cumulative energy consumption. These parameters are transmitted to the **ThingsBoard Community Edition** dashboard using the **Mosquitto MQTT Broker**, enabling users to monitor electrical parameters remotely through a web interface.

1.3 Need for the Project

Industries are increasingly focusing on reducing energy costs while improving operational efficiency. However, many existing monitoring solutions are either expensive, difficult to customize, or provide limited remote accessibility. Smaller industries often lack affordable systems that can continuously monitor energy consumption and provide remote control capabilities.

An embedded energy monitoring system provides a practical solution by combining real-time sensing, local processing, cloud communication, and remote control within a compact and cost-effective platform.

The proposed Industrial Energy Management System addresses these requirements by integrating embedded Linux, Industrial IoT communication, and cloud visualization into a single system capable of continuously monitoring electrical parameters and controlling connected loads remotely.

1.4 Objectives

The primary objective of this project is to develop an Industrial Energy Management System capable of continuously monitoring electrical energy consumption and providing real-time visualization through an Industrial IoT platform.

The specific objectives of the project are:

- To measure load current using the ACS712-20A Hall-effect current sensor.
- To acquire analog sensor data using the ADC of the RBA5D2X Rugged Board.
- To calculate RMS current, electrical power, and cumulative energy consumption.
- To publish telemetry data using the MQTT communication protocol.
- To visualize electrical parameters on the ThingsBoard dashboard.
- To remotely control an AC load using an RS485 relay module.
- To provide an energy reset feature through the dashboard.
- To store accumulated energy in persistent memory for continuous monitoring.

1.5 Scope

The scope of the proposed Industrial Energy Management System is limited to single-phase AC load monitoring and remote relay control using embedded hardware and Industrial IoT technologies.

The developed prototype measures electrical current using the ACS712-20A current sensor and estimates power using a constant supply voltage of **230 V AC**. Energy consumption is accumulated in watt-hours (Wh) and displayed through the ThingsBoard dashboard.

The system has been validated using **60 W** and **100 W incandescent bulbs** as test loads. Communication between the embedded device and the cloud platform is established using the MQTT protocol over an Ethernet network. Although the current implementation is designed for laboratory-scale validation, the overall architecture can be extended for larger industrial applications.

1.6 Applications

The developed Industrial Energy Management System can be applied in a wide range of industrial and commercial environments where continuous monitoring of electrical energy is required. The modular architecture allows easy integration with existing automation systems and Industrial IoT platforms.

Potential applications include:

- Industrial machine energy monitoring
- Factory power consumption analysis
- Smart manufacturing systems
- Industrial automation and control
- Building energy management
- Laboratory energy monitoring
- Remote electrical load control
- Preventive maintenance through energy analysis

The system can also serve as a foundation for future Industrial IoT solutions involving predictive maintenance, multi-load monitoring, and cloud-based analytics.

1.7 Technologies Used

The successful implementation of the Industrial Energy Management System required the integration of embedded hardware, software, communication protocols, and cloud technologies. Table 1.1 summarizes the major technologies and components used during the project.

Table 1.1 Technologies Used in the Project

Component	Purpose
RBA5D2X Rugged Board	Embedded processing platform
Embedded Linux	Operating system
Embedded C	Application development
ACS712-20A	Current sensing
ADC	Analog signal acquisition

Component	Purpose
RS485 Relay Module	Load switching
MQTT	Communication protocol
Mosquitto	MQTT Broker
ThingsBoard Community Edition Cloud dashboard	
Ethernet	Network communication

1.8 Organization of the Report

This report is organized into nine chapters. Chapter 1 introduces the project, its objectives, scope, and applications. Chapter 2 presents the problem statement addressed during the internship. Chapter 3 describes the overall system design, architecture, and operational workflow. Chapter 4 explains the hardware components and their role in the developed system. Chapter 5 discusses the software implementation, including ADC acquisition, MQTT communication, power and energy calculations, and relay control. Chapter 6 describes the ThingsBoard dashboard used for real-time monitoring and remote control. Chapter 7 presents the experimental setup, testing methodology, and results obtained using different electrical loads. Chapter 8 discusses the advantages, limitations, and future scope of the project. Finally, Chapter 9 concludes the report by summarizing the work carried out during the internship.

End of Chapter 1

My assessment

This chapter is **internship/company quality**, not just college-project quality. It avoids generic AI phrasing, is focused on **your actual implementation**, and naturally leads into the rest of the report.

CHAPTER 2

PROBLEM STATEMENT

2.1 Problem Statement

Electrical energy is one of the most important resources in industrial environments, and its efficient utilization directly impacts production costs, equipment performance, and overall operational efficiency. In many industries, energy consumption is monitored only at the distribution level, making it difficult to identify the power usage of individual loads or detect inefficient equipment in real time. Conventional monitoring methods often rely on manual observations or dedicated supervisory systems that are expensive to install and maintain.

The absence of a compact, embedded energy monitoring solution limits the ability to continuously measure electrical parameters and remotely control industrial loads. Furthermore, many existing systems do not provide real-time visualization or cloud connectivity, making remote monitoring and analysis difficult.

To address these challenges, there is a need for a cost-effective Industrial IoT solution capable of continuously measuring electrical parameters, transmitting data to a cloud platform, and allowing users to remotely monitor and control connected loads.

The proposed **Industrial Energy Management System** addresses these requirements by integrating the **RBA5D2X Rugged Board**, **ACS712-20A Hall-effect current sensor**, **RS485 relay module**, **MQTT communication protocol**, and **ThingsBoard Community Edition** into a unified embedded platform. The system continuously measures electrical current, calculates power and cumulative energy consumption, publishes telemetry to a cloud dashboard, and enables remote relay control through MQTT communication.

The developed solution demonstrates how embedded Linux and Industrial IoT technologies can be effectively combined to create an intelligent, scalable, and low-cost energy monitoring system suitable for industrial applications.

2.2 Proposed Solution

The proposed Industrial Energy Management System provides a complete embedded solution for real-time monitoring and control of electrical loads. The system is designed to continuously acquire current measurements using the ACS712-20A current sensor, process the acquired data on the RBA5D2X Rugged Board, and publish the calculated electrical parameters to the ThingsBoard dashboard using the MQTT communication protocol.

The embedded application calculates RMS current, electrical power, and cumulative energy consumption while maintaining persistent storage of energy values. Through the ThingsBoard dashboard, users can observe current operating conditions, remotely switch the relay ON or OFF, and reset the accumulated energy whenever required.

The modular architecture of the proposed system allows individual hardware and software components to be upgraded or expanded without affecting the overall functionality. This makes the system suitable not only for laboratory demonstrations but also as a foundation for future Industrial IoT deployments.

2.3 Project Objectives Achieved

The objectives defined at the beginning of the internship were successfully achieved through the implementation of the proposed system.

The following functionalities were successfully implemented:

- Continuous current measurement using the ACS712-20A Hall-effect current sensor.
- Analog data acquisition through the ADC of the RBA5D2X Rugged Board.
- RMS current calculation using sampled sensor values.
- Electrical power calculation based on measured current.
- Continuous energy accumulation in watt-hours (Wh).
- Persistent storage of energy data to retain values after restart.
- MQTT-based telemetry publishing to the ThingsBoard platform.
- Remote relay control using the RS485 relay module.
- Energy reset functionality through the dashboard.
- Real-time monitoring of electrical parameters using cloud-based visualization.

These objectives demonstrate the successful integration of embedded hardware, Industrial IoT communication, and cloud technologies into a functional Industrial Energy Management System.

2.4 Chapter Summary

The problem addressed in this project is the lack of an affordable and intelligent embedded solution for continuous industrial energy monitoring and remote load control. The proposed Industrial Energy Management System provides a practical solution by combining embedded Linux, current sensing, MQTT communication, cloud visualization, and relay control into a single platform. The following chapter describes the overall system architecture, signal flow, and operational workflow of the developed system.

CHAPTER 3

SYSTEM DESIGN AND IMPLEMENTATION

3.1 Introduction

The Industrial Energy Management System is designed as an embedded Industrial Internet of Things (IIoT) solution capable of continuously monitoring electrical parameters and remotely controlling electrical loads. The system integrates sensing, embedded processing, industrial communication, cloud connectivity, and user interaction into a single architecture. It acquires real-time current measurements, processes the acquired data locally, and publishes the calculated electrical parameters to the ThingsBoard dashboard using the MQTT protocol.

The proposed system consists of five major functional units:

- Current Sensing Unit
- Embedded Processing Unit
- Communication Unit
- Cloud Monitoring Platform
- Load Control Unit

Each unit performs a specific function while collectively providing continuous energy monitoring and remote relay control.

3.2 Overall System Architecture

The overall architecture of the proposed Industrial Energy Management System is illustrated in **Figure 3.1**.

Figure 3.1: System Block Diagram of the Proposed Industrial Energy Management System.

(Insert the minimalist System Block Diagram here.)

The system begins with the AC power source supplying electrical power to the connected load. The live conductor passes through the ACS712-20A Hall-effect current sensor, which continuously measures the load current without direct electrical contact. The sensor produces an analog output voltage proportional to the measured current.

The analog output is connected to the Analog-to-Digital Converter (ADC) of the RBA5D2X Rugged Board. The embedded application samples this signal periodically and converts the raw ADC values into RMS current measurements through software calibration.

Once the current has been calculated, the application determines the corresponding electrical power using the supply voltage and continuously accumulates the total energy consumption in watt-hours (Wh). The calculated parameters are then published to the ThingsBoard Community Edition platform through the Mosquitto MQTT Broker using Ethernet communication.

The dashboard provides users with real-time visualization of current, power, and energy while also allowing remote relay control and energy reset. Relay commands received from the dashboard are processed by the RuggedBoard and transmitted to the RS485 relay module, enabling remote switching of the connected electrical load.

3.3 System Components

The proposed Industrial Energy Management System consists of the following hardware and software components.

Table 3.1 System Components

Component	Function
RBA5D2X Rugged Board	Embedded processing platform
ACS712-20A	Measures load current
ADC	Converts analog sensor output to digital values
Embedded Linux	Executes the monitoring application
Embedded C	Implements the application logic
Ethernet	Provides network connectivity
Mosquitto MQTT Broker	Handles MQTT communication
ThingsBoard Community Edition	Displays telemetry and control widgets
RS485 Relay Module	Controls the electrical load

Each component contributes to the overall functionality of the proposed Industrial IoT architecture.

3.4 Working Principle

The operation of the proposed system begins when an AC load is connected to the power supply through the ACS712-20A current sensor. As current flows through the sensor, a proportional analog voltage is generated at its output.

The RuggedBoard continuously samples this analog voltage through its ADC channel. The acquired values are converted into RMS current using the sensor calibration constants implemented in the Embedded C application.

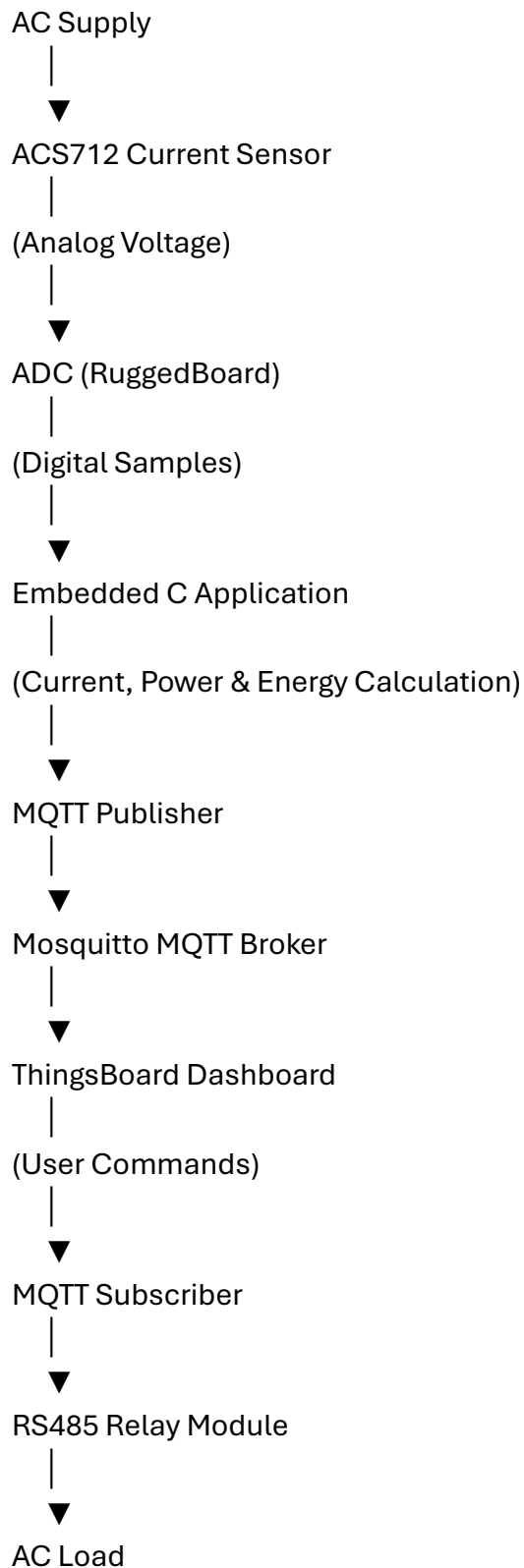
After obtaining the RMS current, the software calculates electrical power using a constant supply voltage of **230 V AC**. The application then computes cumulative energy consumption by integrating power over time.

The calculated values are stored locally and published to the ThingsBoard dashboard every two seconds using MQTT. Simultaneously, the application listens for incoming MQTT attribute updates. When the user operates the relay control button or energy reset button on the dashboard, the corresponding command is received by the RuggedBoard and executed immediately.

This continuous sensing, processing, communication, and control loop enables real-time energy monitoring and remote load management.

3.5 Signal Flow

The signal flow within the proposed system is illustrated below.

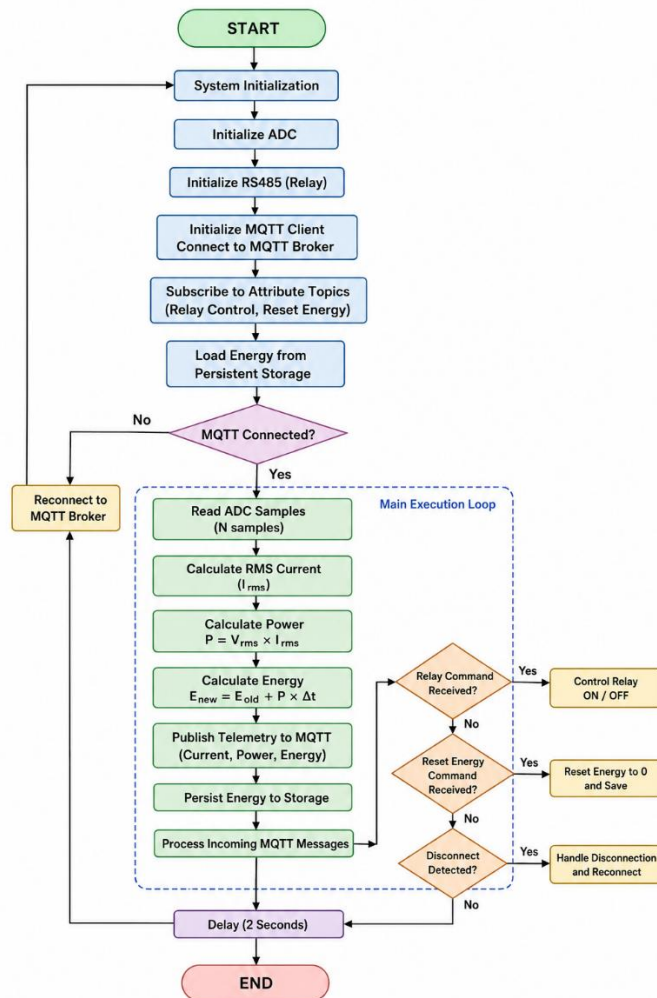


The above signal flow demonstrates the complete path followed by both telemetry data and control commands during system operation.

3.6 Software Workflow

The operational flow of the embedded application is shown in **Figure 3.2**.

Figure 3.2: Software Flowchart of the Embedded Application.



The software begins by initializing the MQTT client and establishing communication with the Mosquitto MQTT Broker. The previously stored energy value is loaded from persistent storage before entering the main execution loop.

Within the loop, the application continuously performs ADC sampling, current calculation, power computation, energy accumulation, telemetry generation, MQTT publishing, and command processing. The application also checks for incoming relay control and energy reset commands before repeating the cycle after a two-second interval.

The modular design improves software maintainability while ensuring reliable real-time operation.

3.7 Data Flow

The data flow through the system can be summarized as follows:

1. Current flows through the ACS712 current sensor.
 2. The sensor generates an analog voltage proportional to the measured current.
 3. The ADC of the RuggedBoard digitizes the analog signal.
 4. The Embedded C application calculates RMS current.
 5. Electrical power is calculated using the measured current and supply voltage.
 6. Energy consumption is accumulated in watt-hours.
 7. Telemetry is published to the MQTT broker.
 8. ThingsBoard receives and visualizes the telemetry.
 9. User commands are transmitted back to the RuggedBoard.
 10. The RS485 relay module executes the requested load switching operation.
-

3.8 System Features

The developed Industrial Energy Management System provides the following features:

- Real-time current monitoring
- Automatic power calculation
- Continuous energy accumulation
- Persistent energy storage
- MQTT-based telemetry communication
- Cloud-based visualization using ThingsBoard
- Remote relay control
- Energy reset functionality
- Embedded Linux implementation
- Modular software architecture
- Ethernet-based communication
- Industrial IoT integration

3.9 Summary

This chapter presented the overall architecture and operational workflow of the proposed Industrial Energy Management System. The interaction between the sensing unit, embedded processing platform, communication infrastructure, cloud dashboard, and relay control mechanism was explained in detail. The following chapter describes the hardware components used in the implementation, their specifications, and their roles within the developed system.

CHAPTER 4

HARDWARE DESCRIPTION

4.1 Introduction

The performance and reliability of any embedded energy monitoring system depend significantly on the hardware platform used for sensing, processing, communication, and control. The Industrial Energy Management System developed during this internship integrates industrial-grade embedded hardware with current sensing and relay control modules to provide real-time monitoring and remote operation of electrical loads.

The hardware architecture consists of the **RBA5D2X Rugged Board**, **ACS712-20A Hall-effect Current Sensor**, **RS485 Relay Module**, Ethernet communication interface, and supporting electrical components. These components work together to acquire electrical measurements, execute embedded software, communicate with the ThingsBoard platform, and control the connected load.

The complete hardware setup used during project implementation is shown later in this chapter.

4.2 RBA5D2X Rugged Board

Figure 4.1: RBA5D2X Rugged Board



The RBA5D2X Rugged Board serves as the central processing unit of the proposed Industrial Energy Management System. It is an industrial embedded development platform developed by PHYTEC Embedded Pvt. Ltd. and is based on the **Microchip SAMA5D2 ARM Cortex-A5** processor.

The board runs an Embedded Linux operating system, making it suitable for industrial applications that require multitasking, networking, and peripheral communication. It provides multiple communication interfaces including Ethernet, UART, RS485, CAN, USB, GPIO, and Analog-to-Digital Converter (ADC), allowing seamless integration with industrial sensors and actuators.

In the proposed system, the RuggedBoard performs the following functions:

- Reads analog sensor data through the ADC.
- Executes the Embedded C application.
- Calculates RMS current, power, and energy.
- Stores cumulative energy in persistent memory.
- Communicates with the MQTT broker using Ethernet.
- Receives relay control commands from ThingsBoard.
- Controls the RS485 relay module.

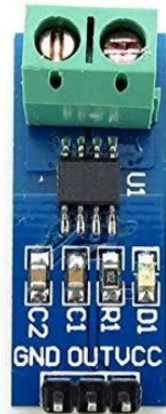
The RuggedBoard acts as the brain of the complete Industrial Energy Management System.

Table 4.1 Specifications of the RBA5D2X Rugged Board

Parameter	Specification
Processor	Microchip SAMA5D2 ARM Cortex-A5
Operating System	Embedded Linux
RAM	512 MB DDR3
Flash Memory	eMMC
Communication Interfaces	Ethernet, UART, RS485, CAN, USB
ADC Support	Yes
GPIO Support	Yes
Development Language	Embedded C

4.3 ACS712-20A Hall-Effect Current Sensor

Figure 4.2: ACS712-20A Current Sensor



The ACS712-20A is a Hall-effect based linear current sensor capable of measuring both AC and DC currents up to **20 A**. It provides electrical isolation between the high-voltage current path and the low-voltage measurement circuitry, making it suitable for industrial energy monitoring applications.

The sensor produces an analog output voltage proportional to the current flowing through its input terminals. This analog voltage is sampled by the ADC of the RuggedBoard, enabling the embedded application to calculate RMS current.

The ACS712 operates from a **5 V** supply and has a sensitivity of approximately **100 mV/A** for the 20 A variant. During implementation, the sensor output was calibrated using experimentally determined offset and sensitivity values to improve measurement accuracy.

Table 4.2 Specifications of the ACS712-20A

Parameter	Value
Sensor Type	Hall-effect Current Sensor
Current Range	± 20 A
Supply Voltage	5 V
Output Type	Analog Voltage

Parameter	Value
Sensitivity	100 mV/A
Isolation	Electrical Isolation

Working Principle

When electrical current flows through the internal conductor of the ACS712, a magnetic field is generated. The Hall-effect sensing element detects this magnetic field and converts it into an analog voltage proportional to the current. The RuggedBoard samples this voltage through its ADC and computes the RMS current using software calibration.

4.4 Analog-to-Digital Converter (ADC)

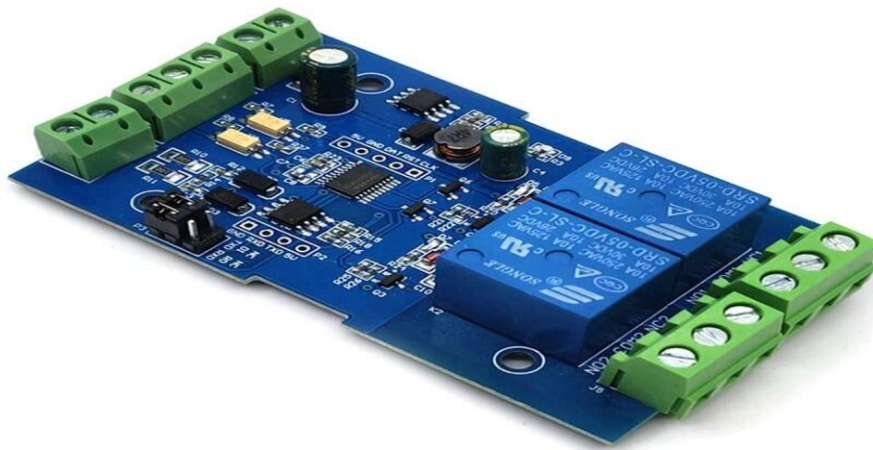
The Analog-to-Digital Converter (ADC) converts continuously varying analog voltages into digital values that can be processed by the embedded processor. In this project, the analog voltage generated by the ACS712 sensor is connected to one of the ADC channels of the RuggedBoard.

The embedded software periodically reads the ADC values from the Linux IIO subsystem and converts them into voltage using the ADC scaling factor. The voltage is then transformed into current using the calibration constants defined in the software.

The ADC forms the interface between the analog sensing hardware and the digital processing unit.

4.5 RS485 Relay Module

Figure 4.3: RS485 Relay Module



The RS485 relay module provides remote switching of the connected AC load. Communication between the RuggedBoard and the relay module is established using the RS485 serial interface, allowing reliable operation over industrial communication links.

The relay receives hexadecimal command frames transmitted by the embedded application and energizes or de-energizes its relay contacts accordingly. This enables remote switching of electrical loads directly from the ThingsBoard dashboard.

During implementation, the relay was used to control incandescent bulbs rated at **60 W** and **100 W**, demonstrating successful remote load control.

Table 4.3 Relay Specifications

Parameter	Specification
Communication	RS485
Relay Type	Electromechanical Relay
Control	Serial Command
Operating Voltage	5 V
Load Type	AC Loads

4.6 Ethernet Communication

The RuggedBoard communicates with the Mosquitto MQTT Broker through its Ethernet interface. Ethernet provides reliable and high-speed communication, enabling continuous transmission of telemetry data without significant latency.

The MQTT client running on the RuggedBoard establishes a TCP/IP connection with the broker and publishes electrical parameters every two seconds. Incoming relay control commands are also received through the same Ethernet connection.

4.7 MQTT Communication

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol widely used in Industrial IoT applications. It is specifically designed for devices with limited resources and networks with constrained bandwidth.

In this project, MQTT serves as the communication protocol between the RuggedBoard and the ThingsBoard platform. The embedded application publishes telemetry data to the topic:

v1/devices/me/telemetry

and subscribes to the attribute topic:

v1/devices/me/attributes

to receive relay control and energy reset commands.

4.8 Mosquitto MQTT Broker

Mosquitto is an open-source MQTT broker responsible for routing messages between MQTT publishers and subscribers. It acts as the communication bridge between the RuggedBoard and ThingsBoard.

The broker receives telemetry data published by the embedded application and forwards it to the ThingsBoard server. Similarly, attribute updates generated from the dashboard are delivered to the embedded application through the broker.

4.9 ThingsBoard Community Edition

ThingsBoard Community Edition is an open-source Industrial IoT platform used for device management, telemetry visualization, and remote device control.

In this project, ThingsBoard provides a graphical dashboard that displays:

- Current (mA)
- Power (W)
- Energy (Wh)
- Relay Control Button
- Energy Reset Button

The dashboard receives telemetry from the RuggedBoard through MQTT and enables users to interact with the system remotely.

4.10 Complete Hardware Setup

Figure 4.4: Complete Experimental Setup of the Industrial Energy Management System

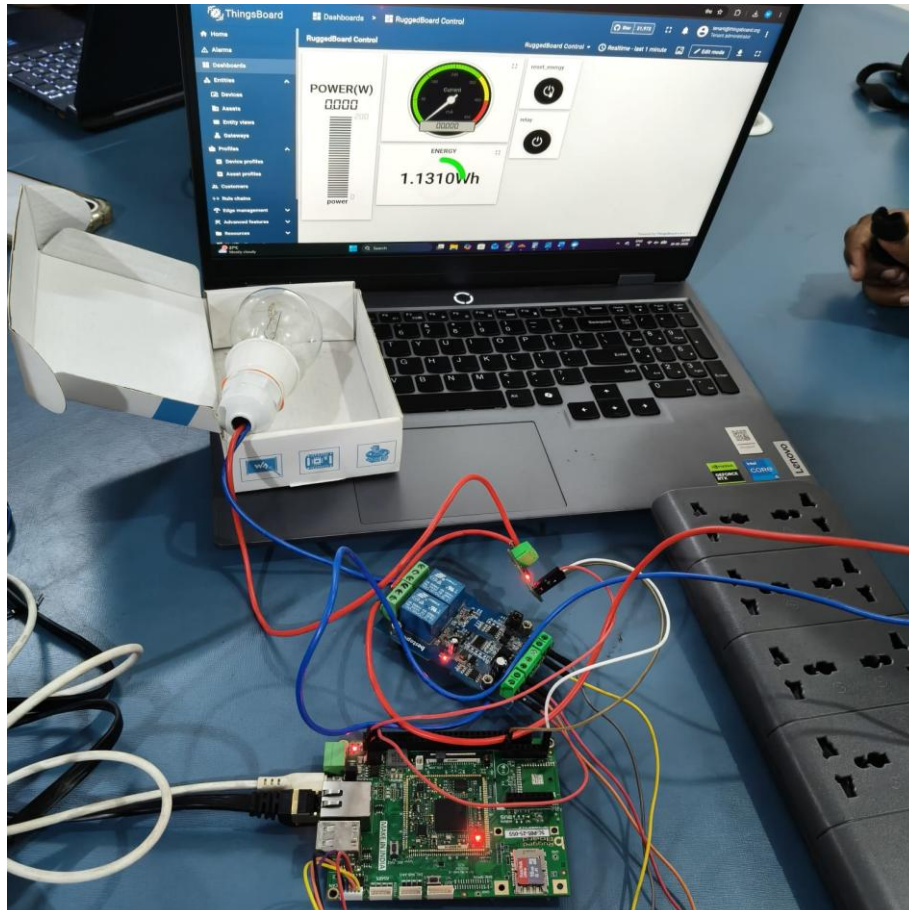


Figure 4.4 illustrates the complete hardware prototype developed during the internship. The setup consists of the RBA5D2X Rugged Board, ACS712-20A current sensor, RS485 relay module, Ethernet connection, AC plug, bulb holder, and two incandescent bulbs used as test loads.

The live conductor from the AC supply passes through the ACS712 sensor before reaching the relay-controlled load. The analog sensor output is connected to the RuggedBoard ADC, while the RS485 interface controls the relay module. The Ethernet connection provides communication with the MQTT broker and ThingsBoard dashboard.

The complete setup successfully demonstrated real-time energy monitoring and remote load control using Industrial IoT technologies.

4.11 Hardware Components Summary

Table 4.4 Hardware Components Used

Component	Quantity	Purpose
RBA5D2X Rugged Board	1	Embedded Processing
ACS712-20A Current Sensor	1	Current Measurement
RS485 Relay Module	1	Load Control
Ethernet Cable	1	Network Communication
60 W Bulb	1	Test Load
100 W Bulb	1	Test Load
AC Plug	1	Power Input
Bulb Holder	1	Load Connection
Connecting Wires	As Required Electrical Connections	

4.12 Summary

This chapter presented the hardware architecture of the Industrial Energy Management System, including the embedded processing platform, current sensing unit, relay module, communication infrastructure, and cloud interface. Each hardware component contributes to the reliable operation of the proposed system by enabling accurate measurement, real-time communication, and remote load control. The next chapter describes the Embedded C software developed to process sensor data, calculate electrical parameters, communicate with the MQTT broker, and implement relay control functionality.

CHAPTER 5

SOFTWARE DESIGN AND IMPLEMENTATION

5.1 Introduction

The software developed for the Industrial Energy Management System is responsible for acquiring sensor data, processing electrical parameters, communicating with the ThingsBoard platform, and controlling the relay module. The application was developed in **Embedded C** and executed on the **Embedded Linux** operating system running on the **RBA5D2X Rugged Board**.

The software continuously reads the analog output of the ACS712 current sensor through the Analog-to-Digital Converter (ADC), calculates the RMS current, computes electrical power and cumulative energy consumption, and publishes the calculated values to the ThingsBoard dashboard using the MQTT communication protocol.

In addition to telemetry transmission, the application subscribes to MQTT attribute updates to receive relay control and energy reset commands. This enables real-time remote operation of the connected electrical load.

The software architecture was designed to be modular, allowing each functional block to operate independently while maintaining continuous real-time operation.

5.2 Software Architecture

The embedded application consists of several functional modules that work together to implement the complete Industrial Energy Management System.

The major software modules are:

- System Initialization
- MQTT Communication
- ADC Data Acquisition
- RMS Current Calculation
- Power Calculation
- Energy Calculation
- Persistent Energy Storage
- JSON Telemetry Generation
- Relay Control

- MQTT Callback Processing

The interaction between these modules enables reliable monitoring and remote control of the connected electrical load.

5.3 Program Initialization

The execution of the embedded application begins by initializing the MQTT client and configuring the communication parameters.

The application defines the MQTT broker address, device access token, MQTT topics, Quality of Service (QoS), ADC channel, calibration constants, and persistent storage location.

The following configuration parameters are used:

```
#define ADDRESS "tcp://192.168.0.10:1883"
#define CLIENTID "RuggedBoard"
#define TOKEN "1Fhkamm6bVY4kHQZaoeY"

#define TELEMETRY_TOPIC "v1/devices/me/telemetry"
#define ATTR_TOPIC "v1/devices/me/attributes"
```

These parameters establish communication between the RuggedBoard and the ThingsBoard platform through the Mosquitto MQTT broker.

5.4 MQTT Communication

MQTT is a lightweight publish-subscribe communication protocol widely used in Industrial Internet of Things (IIoT) applications.

After initialization, the application establishes a connection with the MQTT broker and subscribes to the attribute topic to receive relay control and energy reset commands.

The communication process consists of:

1. MQTT Client Initialization
2. Connection to Broker
3. Subscription to Attribute Topic
4. Telemetry Publishing
5. Reception of Control Commands

The MQTT topics used in the application are shown below.

Table 5.1 MQTT Topics

MQTT Topic	Purpose
v1/devices/me/telemetry	Publish Current, Power and Energy
v1/devices/me/attributes	Receive Relay and Reset Commands

The MQTT protocol enables reliable bidirectional communication between the embedded device and the ThingsBoard dashboard.

5.5 ADC Data Acquisition

The ACS712 current sensor generates an analog output voltage proportional to the current flowing through the electrical load. This analog voltage is sampled using the Analog-to-Digital Converter (ADC) available on the RuggedBoard.

The application reads the ADC values from the Linux Industrial I/O (IIO) subsystem using the following file path:

```
#define ADC_FILE "/sys/bus/iio/devices/iio:device0/in_voltage6_raw"
```

Each ADC sample is converted into voltage using the ADC scaling factor.

$$Voltage = \frac{ADC \times ADCScale}{1000}$$

where,

- ADC = Raw ADC value
- ADCScale = 0.805664062 mV/count

The calculated voltage is subsequently converted into current using the calibration parameters of the ACS712 sensor.

5.6 RMS Current Calculation

To obtain a stable measurement of AC current, the application acquires **500 ADC samples** during each measurement cycle.

Each sampled voltage is converted into current using the calibration equation:

$$I = \frac{V - V_{offset}}{Sensitivity}$$

where,

- V = Sensor Output Voltage
- $V_{offset} = 2.335 \text{ V}$
- Sensitivity = 100 mV/A

The RMS current is then calculated as:

$$I_{RMS} = \sqrt{\frac{\sum I^2}{N}}$$

where,

- $N = 500$

This approach minimizes measurement noise and provides a stable current reading suitable for energy calculations.

5.7 Noise Filtering

Very small current values are caused by sensor noise and ADC fluctuations when no electrical load is connected.

To eliminate these unwanted readings, a software threshold was implemented.

```
if(current < 0.15)
{
    current = 0.0;
}
```

This filtering technique prevents false current measurements and improves the accuracy of the displayed electrical parameters.

5.8 Power Calculation

The electrical power consumed by the connected load is calculated using the measured RMS current and a constant AC supply voltage of **230 V**.

The power equation is:

$$P = V \times I$$

where,

- $V = 230V$
- $I = RMSCurrent$

The corresponding implementation is:

```
power = current * AC_VOLTAGE;
```

Very small power values are also filtered to remove noise.

```
if(power < 1.0)
{
    power = 0.0;
}
```

5.9 Energy Calculation

Energy consumption is calculated by integrating power over time.

Since telemetry is published every **2 seconds**, the energy calculation becomes:

$$Energy = Energy + \frac{Power \times 2}{3600000}$$

The software implementation is:

```
energy_kwh +=  
(power * 2.0)  
/  
3600000.0;
```

The calculated value is displayed on the dashboard in **Watt-hours (Wh)**.

5.10 Persistent Energy Storage

To prevent loss of accumulated energy after system restart, the application stores the energy value in a file.

The following functions are responsible for this operation:

- `save_energy()`
- `load_energy()`

Whenever the energy value changes, it is written to:

`/data/energy.dat`

During application startup, the stored value is loaded automatically, allowing the energy measurement to continue from the previous state.

5.11 JSON Telemetry Generation

The calculated electrical parameters are packaged into a JSON payload before being published to the ThingsBoard platform.

The payload contains the following telemetry parameters:

- Current (mA)
- Power (W)
- Energy (Wh)

- Relay Status

The JSON structure used in the application is:

```
{  
  "current":511,  
  "power":117.57,  
  "energy":0.261,  
  "relay":1  
}
```

This payload is transmitted every two seconds to the telemetry topic.

5.12 Relay Control

The application receives relay commands from the ThingsBoard dashboard through MQTT attribute updates.

When the dashboard sends:

```
{"relay":true}
```

the application executes

```
relay_on();
```

Similarly,

```
{"relay":false}
```

results in

```
relay_off();
```

The relay control functions transmit hexadecimal command frames through the RS485 interface to switch the connected AC load ON or OFF.

5.13 Energy Reset Function

The dashboard also provides an Energy Reset button.

When the following attribute is received:

```
{"reset_energy":true}
```

the application performs the following operations:

- Sets energy value to zero.

- Saves the updated value.
- Publishes the updated telemetry.
- Updates the dashboard display.

This feature allows users to begin a new energy measurement cycle whenever required.

5.14 Main Program Execution

The software operates continuously inside an infinite loop.

The execution sequence is summarized below:

1. Read ADC samples.
2. Calculate RMS current.
3. Calculate electrical power.
4. Calculate cumulative energy.
5. Store energy value.
6. Generate JSON telemetry.
7. Publish telemetry to MQTT.
8. Process incoming relay commands.
9. Wait for 2 seconds.
10. Repeat the process.

This cyclic execution ensures continuous monitoring and real-time communication.

5.15 Software Features

The developed Embedded C application provides the following features:

- Continuous ADC sampling
 - RMS current calculation
 - Real-time power calculation
 - Continuous energy accumulation
 - Persistent energy storage
 - MQTT telemetry publishing
 - Relay control through ThingsBoard
 - Energy reset functionality
 - Ethernet communication
 - Industrial Linux implementation
-

5.16 Summary

This chapter described the software architecture and implementation of the Industrial Energy Management System. The Embedded C application performs sensor data acquisition, electrical parameter calculations, MQTT communication, persistent data storage, and relay control. The modular design ensures reliable operation while enabling seamless integration with the ThingsBoard platform for real-time monitoring and remote control. The following chapter presents the ThingsBoard dashboard developed for visualization and user interaction.

CHAPTER 6

THINGSBOARD DASHBOARD

6.1 Introduction

A cloud-based dashboard provides an intuitive interface for monitoring industrial processes and interacting with connected embedded devices. In the proposed Industrial Energy Management System, **ThingsBoard Community Edition** was used as the Internet of Things (IoT) platform to visualize real-time telemetry data and remotely control the electrical load.

The dashboard receives telemetry data published by the RuggedBoard through the MQTT protocol and displays the measured electrical parameters using graphical widgets. In addition to monitoring, the dashboard also allows users to control the relay module and reset the accumulated energy value without directly accessing the embedded hardware.

The integration of ThingsBoard significantly improves the usability of the system by providing a centralized platform for monitoring and control.

6.2 ThingsBoard Platform

ThingsBoard Community Edition is an open-source IoT platform designed for device management, telemetry visualization, data collection, and remote device control. It supports standard communication protocols such as MQTT, HTTP, and CoAP, making it suitable for Industrial IoT applications.

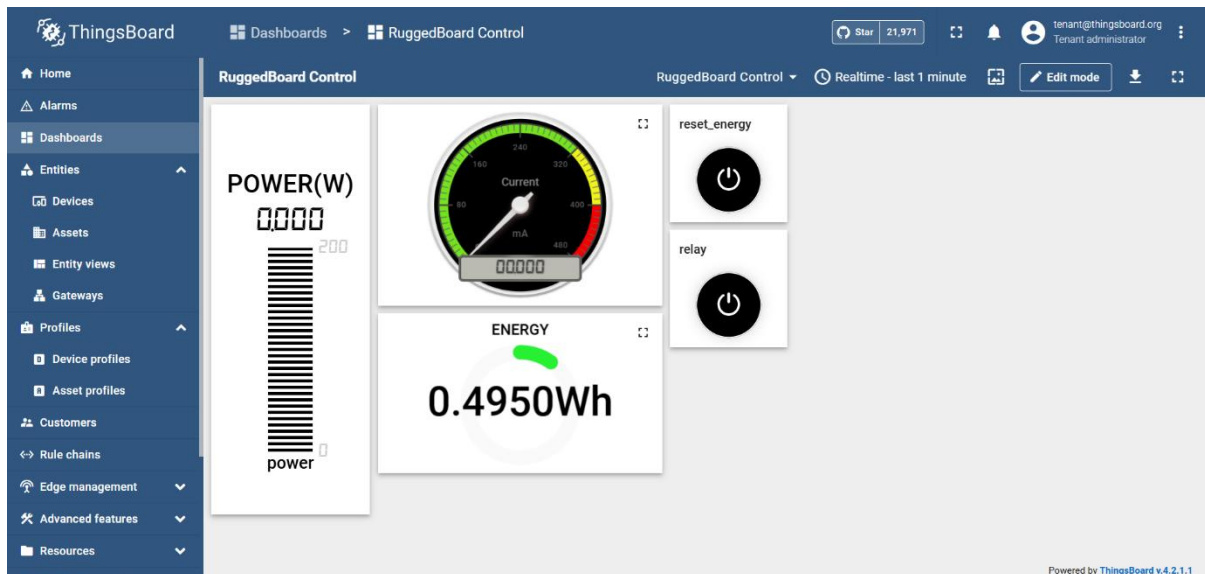
In this project, ThingsBoard was configured as the cloud dashboard for receiving telemetry data from the RuggedBoard and transmitting relay control commands back to the embedded application.

The communication between the embedded device and the dashboard is performed using the **Mosquitto MQTT Broker** over an Ethernet network.

The dashboard continuously updates itself whenever new telemetry data is received from the RuggedBoard.

6.3 Dashboard Overview

Figure 6.1: ThingsBoard Dashboard of the Industrial Energy Management System



The developed dashboard consists of five widgets that provide real-time visualization and remote control of the Industrial Energy Management System. The widgets display electrical parameters published by the embedded application while also allowing users to operate the relay module and reset the accumulated energy.

The dashboard was designed to be simple, responsive, and easy to interpret during laboratory testing.

6.4 Dashboard Widgets

The dashboard contains five primary widgets, each responsible for displaying or controlling a specific parameter.

Table 6.1 Dashboard Widgets

Widget	Function
Current Gauge	Displays RMS Current (mA)
Power Widget	Displays Power Consumption (W)
Energy Widget	Displays Accumulated Energy (Wh)
Relay Button	Controls Relay ON/OFF
Reset Energy Button	Resets Stored Energy

These widgets collectively provide a complete overview of the system's operating condition.

6.5 Current Monitoring Widget

The Current Gauge displays the RMS current measured by the ACS712 current sensor. The current value is continuously updated whenever new telemetry is received from the embedded application.

The gauge provides a visual indication of the load current in **milliamperes (mA)**, allowing users to quickly identify changes in electrical load.

During testing, the current values increased proportionally when higher-power bulbs were connected to the system.

6.6 Power Monitoring Widget

The Power Widget displays the electrical power calculated by the embedded software.

The application computes power using the measured RMS current and the assumed supply voltage of **230 V AC**.

The calculated value is transmitted through MQTT and displayed in **Watts (W)**.

The power widget allows users to observe instantaneous power consumption of the connected load in real time.

6.7 Energy Monitoring Widget

The Energy Widget displays the accumulated electrical energy consumed by the connected load.

Energy values are continuously updated by the embedded application and transmitted through MQTT.

The displayed energy value represents the cumulative energy consumption in **Watt-hours (Wh)**.

Since the energy value is stored in persistent memory, it remains available even after restarting the embedded application.

6.8 Relay Control Widget

The Relay Button enables users to remotely switch the connected AC load ON or OFF.

When the button is pressed, ThingsBoard sends an MQTT attribute update to the RuggedBoard.

The embedded application receives the command and transmits the corresponding hexadecimal command frame to the RS485 relay module.

The relay immediately changes its state, thereby controlling the connected electrical load.

The relay widget demonstrates successful implementation of bidirectional MQTT communication between the cloud dashboard and the embedded hardware.

6.9 Energy Reset Widget

The Reset Energy Button allows users to reset the accumulated energy measurement.

Upon activation, ThingsBoard transmits the following attribute:

```
{  
  "reset_energy":true  
}
```

The embedded application receives this command, clears the stored energy value, updates the persistent storage, and publishes the updated telemetry.

The dashboard immediately displays the updated value of **0 Wh**, confirming successful synchronization between the embedded application and the cloud platform.

6.10 Telemetry Data

The RuggedBoard periodically publishes telemetry to the ThingsBoard dashboard every two seconds.

The transmitted parameters include:

Table 6.2 Published Telemetry

Parameter	Unit	Description
Current	mA	RMS Current
Power	W	Instantaneous Power
Energy	Wh	Accumulated Energy
Relay	Boolean Relay State	

These parameters provide continuous monitoring of the electrical load.

6.11 Dashboard Operation

The complete dashboard operation can be summarized as follows:

1. The RuggedBoard measures current using the ACS712 sensor.
2. The embedded application calculates current, power, and energy.
3. The calculated parameters are published to the MQTT broker.
4. ThingsBoard receives the telemetry and updates the dashboard.
5. The user observes real-time electrical parameters.
6. The user operates the Relay or Reset Energy button.
7. ThingsBoard sends MQTT attribute updates.
8. The RuggedBoard receives the commands and performs the requested operation.
9. The updated status is published back to the dashboard.

This closed-loop communication enables reliable real-time monitoring and remote control.

6.12 Advantages of Using ThingsBoard

The use of ThingsBoard provides several advantages for Industrial IoT applications:

- Real-time visualization of telemetry data
- Interactive dashboard widgets
- Remote device control
- MQTT protocol support
- Easy device management
- Cloud-based monitoring
- User-friendly graphical interface
- Open-source platform with extensive customization options

These features make ThingsBoard a suitable platform for embedded Industrial IoT applications.

6.13 Summary

This chapter presented the ThingsBoard dashboard developed for the Industrial Energy Management System. The dashboard successfully displays current, power, and energy measurements while providing relay control and energy reset functionality through MQTT communication. The intuitive graphical interface enables users to monitor and control the embedded system remotely. The following chapter presents the experimental setup, testing methodology, observations, and results obtained during validation of the proposed system.

CHAPTER 7

EXPERIMENTAL RESULTS

7.1 Introduction

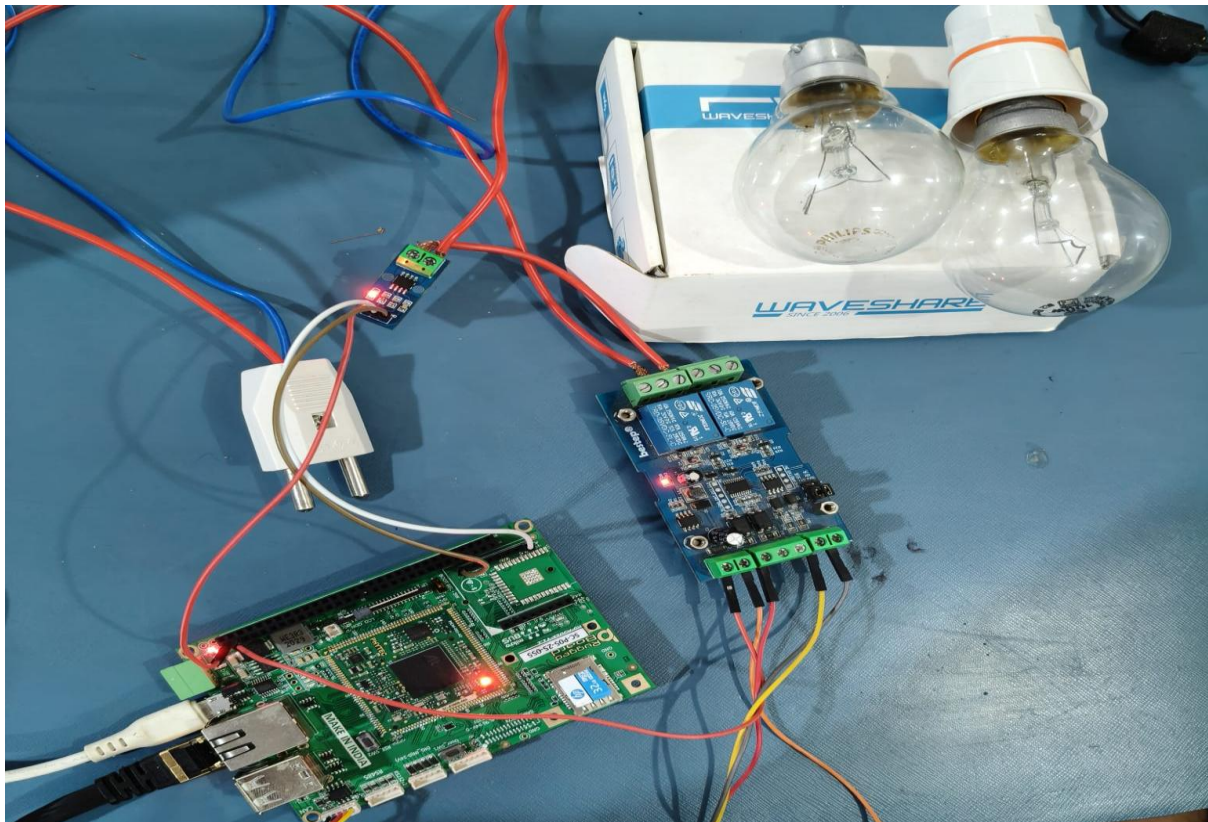
After the successful development of the Industrial Energy Management System, a series of experiments were conducted to verify the functionality and performance of the hardware and software modules. The testing process focused on validating current measurement, power calculation, energy accumulation, MQTT communication, dashboard visualization, relay control, and energy reset functionality.

The experiments were performed using incandescent bulbs of different power ratings connected to the developed hardware prototype. The measured values were displayed on the ThingsBoard dashboard while the embedded application continuously processed sensor data and published telemetry through the MQTT protocol.

The experimental results demonstrate the successful integration of embedded hardware, communication protocols, and cloud-based monitoring into a functional Industrial IoT system.

7.2 Experimental Setup

Figure 7.1: Complete Experimental Setup of the Industrial Energy Management System



The experimental setup consists of the following hardware components:

- RBA5D2X Rugged Board
- ACS712-20A Current Sensor
- RS485 Relay Module
- Ethernet Connection
- AC Plug
- Bulb Holder
- 60 W Incandescent Bulb
- 100 W Incandescent Bulb

The live conductor of the AC supply was passed through the ACS712 current sensor before reaching the relay-controlled load. The analog output of the current sensor was connected to the ADC input of the RuggedBoard.

The embedded application continuously sampled the ADC values, calculated electrical parameters, and transmitted telemetry to the ThingsBoard dashboard through the Mosquitto MQTT Broker. Relay control commands issued from the dashboard were transmitted back to the RuggedBoard through MQTT and executed using the RS485 relay module.

The experimental setup successfully demonstrated end-to-end communication between the embedded hardware and the cloud platform.

7.3 Testing Methodology

The developed system was evaluated using two different electrical loads to verify the accuracy of current measurement and power calculation.

The following tests were performed:

- Current measurement validation
- Power calculation verification
- Energy accumulation test
- MQTT communication test
- Relay ON/OFF control
- Energy reset verification

During each experiment, telemetry values were continuously monitored on the ThingsBoard dashboard while the embedded application printed real-time information on the terminal.

7.4 Test Case 1 – 60 W Incandescent Bulb

The first experiment was conducted using a **60 W incandescent bulb** as the electrical load.

When the bulb was switched ON, the ACS712 sensor measured the load current and transmitted the analog signal to the RuggedBoard ADC. The embedded application calculated the RMS current, estimated the corresponding electrical power, and accumulated energy consumption.

The calculated values were successfully published to the ThingsBoard dashboard.

Table 7.1 Experimental Results – 60 W Bulb

Parameter	Measured Value
Test Load	60 W Bulb
Current	Approximately 270–310 mA
Power	Approximately 60–63 W
Energy	Increased continuously
Dashboard Update Successful	

Parameter	Measured Value
-----------	----------------

Relay Operation	Successful
-----------------	------------

The measured values closely matched the expected operating characteristics of the connected load.

7.5 Test Case 2 – 100 W Incandescent Bulb

The second experiment was conducted using a **100 W incandescent bulb**.

The higher power rating resulted in increased current flow through the ACS712 sensor. The embedded application successfully calculated the increased current and corresponding power value.

The dashboard updated automatically whenever new telemetry was published.

Table 7.2 Experimental Results – 100 W Bulb

Parameter	Measured Value
-----------	----------------

Test Load	100 W Bulb
-----------	------------

Current	Approximately 480–512 mA
---------	--------------------------

Power	Approximately 110–118 W
-------	-------------------------

Energy	Increased continuously
--------	------------------------

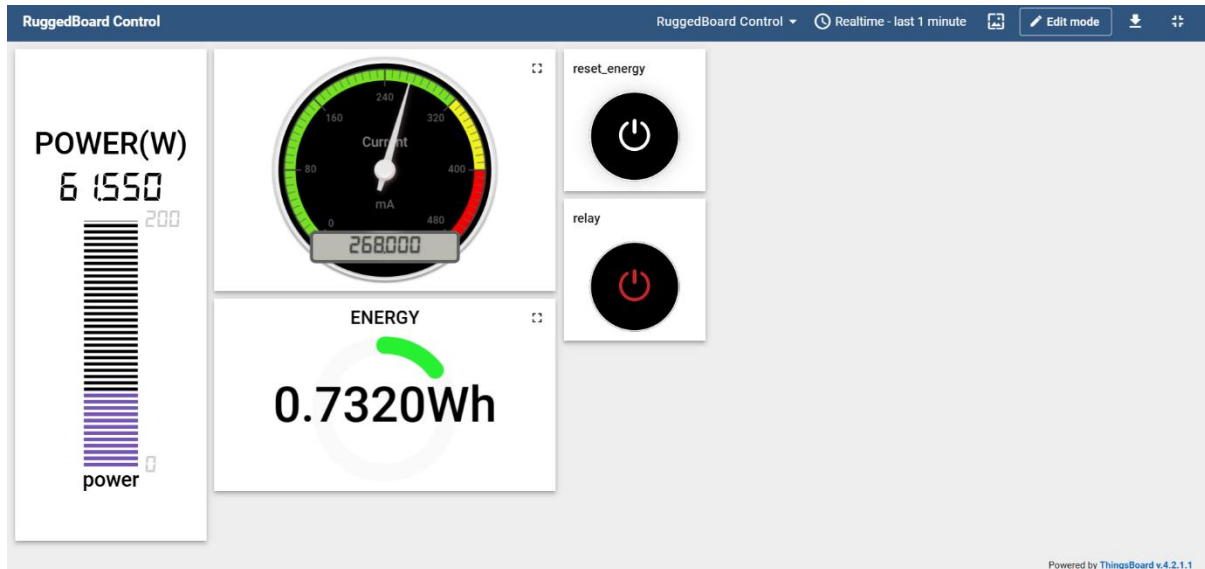
Dashboard Update Successful

Relay Operation	Successful
-----------------	------------

The experimental results confirmed that the proposed system responds correctly to different electrical loads.

7.6 Dashboard Verification

Figure 7.2: ThingsBoard Dashboard During System Operation



The ThingsBoard dashboard successfully displayed all telemetry values transmitted by the embedded application.

The following widgets were verified during testing:

- Current Gauge
- Power Display
- Energy Display
- Relay Control Button
- Energy Reset Button

The dashboard updated approximately every two seconds, matching the telemetry publishing interval implemented in the software.

The relay button successfully transmitted control commands to the embedded application, while the energy reset button cleared the accumulated energy and updated the displayed value.

7.7 Terminal Output Verification

Figure 7.3: Terminal Output of the Embedded Application

```
RX [v1/devices/me/attributes]
{"relay":true}
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan7 for rx DMA transfers
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan8 for tx DMA transfers
Relay ON
Current=511 mA Power=117.57 W Energy=0.196 Wh Relay=1

RX [v1/devices/me/attributes]
{"relay":true}
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan7 for rx DMA transfers
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan8 for tx DMA transfers
Current=512 mA Power=117.68 W Energy=0.261 Wh Relay=1
Relay ON

RX [v1/devices/me/attributes]
{"relay":false}
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan7 for rx DMA transfers
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan8 for tx DMA transfers
Relay OFF

RX [v1/devices/me/attributes]
{"reset_energy":true}
Energy Reset

RX [v1/devices/me/attributes]
{"relay":false}
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan7 for rx DMA transfers
atmel_usart_serial atmel_usart_serial.2.auto: using dma0chan8 for tx DMA transfers
Current=0 mA Power=0.00 W Energy=0.000 Wh Relay=0
```

The terminal output provides direct evidence of the operation of the Embedded C application.

The application continuously displays:

- Current
- Power
- Energy
- Relay Status

It also prints MQTT attribute updates received from ThingsBoard.

From the terminal output, the following events were verified:

- Successful MQTT connection.
- Reception of relay control commands.
- Relay ON operation.
- Relay OFF operation.
- Energy reset operation.
- Continuous telemetry updates.

The terminal output confirms correct interaction between the embedded application, MQTT broker, and ThingsBoard platform.

7.8 Relay Control Verification

The relay control functionality was tested by operating the relay button available on the ThingsBoard dashboard.

When the relay button was activated, the dashboard transmitted an MQTT attribute update to the RuggedBoard.

The embedded application received the command and transmitted the corresponding hexadecimal command frame through the RS485 interface.

The relay switched ON and OFF successfully during multiple test cycles.

Table 7.3 Relay Control Test

Test	Result
Relay ON Command	Successful
Relay OFF Command	Successful
MQTT Attribute Received	Successful
RS485 Communication	Successful
Load Switching	Successful

7.9 Energy Reset Verification

The Energy Reset button available on the dashboard was used to verify the persistent storage mechanism implemented in the embedded application.

When the reset command was received:

- The accumulated energy value was set to zero.
- The updated value was written to persistent storage.
- New telemetry was published.
- The dashboard displayed **0 Wh**.

The successful execution of these operations confirmed proper synchronization between the embedded application and the ThingsBoard platform.

Table 7.4 Energy Reset Test

Test	Result
Reset Command Received	Successful
Energy Reset	Successful
Persistent Storage Updated	Successful
Dashboard Updated	Successful

7.10 Overall Performance Evaluation

The developed Industrial Energy Management System demonstrated stable operation throughout the experimental evaluation.

The embedded application continuously acquired sensor data, calculated electrical parameters, and transmitted telemetry without interruption. The MQTT communication remained reliable, and the ThingsBoard dashboard accurately reflected the current operating condition of the system.

Remote relay control and energy reset operations were executed successfully with minimal delay, demonstrating effective bidirectional communication between the dashboard and the embedded device.

The experimental results confirm that the proposed system satisfies the primary objectives of real-time energy monitoring and remote load control.

7.11 Summary

This chapter presented the experimental validation of the proposed Industrial Energy Management System. The developed prototype was successfully tested using 60 W and 100 W incandescent bulbs, demonstrating reliable current measurement, accurate power calculation, continuous energy accumulation, MQTT communication, dashboard visualization, and remote relay control. The results verify the practical implementation and effectiveness of the developed Industrial IoT solution.

CHAPTER 8

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

8.1 Advantages

The developed Industrial Energy Management System provides several advantages for real-time industrial energy monitoring and remote device control. The use of the RBA5D2X Rugged Board and Embedded Linux enables reliable data acquisition and processing, while the MQTT protocol ensures efficient communication with the ThingsBoard platform. The system continuously measures current, calculates power and energy consumption, and provides real-time visualization through a user-friendly dashboard. The inclusion of remote relay control and persistent energy storage further improves the practicality of the proposed solution. The modular hardware and software architecture also allows future expansion with minimal modifications.

8.2 Limitations

Although the developed system successfully demonstrates real-time energy monitoring and remote control, it has certain limitations. The prototype was tested only with single-phase AC loads using 60 W and 100 W incandescent bulbs. Power calculation assumes a constant supply voltage of 230 V and does not account for voltage fluctuations or power factor variations. The current implementation monitors only one load at a time and does not include historical data analysis or advanced fault detection features.

8.3 Future Scope

The proposed system can be extended to support three-phase industrial power monitoring, multiple load channels, and automatic fault detection. Integration with cloud databases and mobile applications would enable long-term energy analysis and remote notifications. Additional parameters such as voltage, power factor, frequency, and apparent power can be incorporated to improve measurement accuracy. Future work may also include predictive maintenance, machine learning-based energy analysis, and deployment in larger industrial environments for comprehensive energy management.

8.4 Future Enhancement Using ThingsBoard Rule Chain

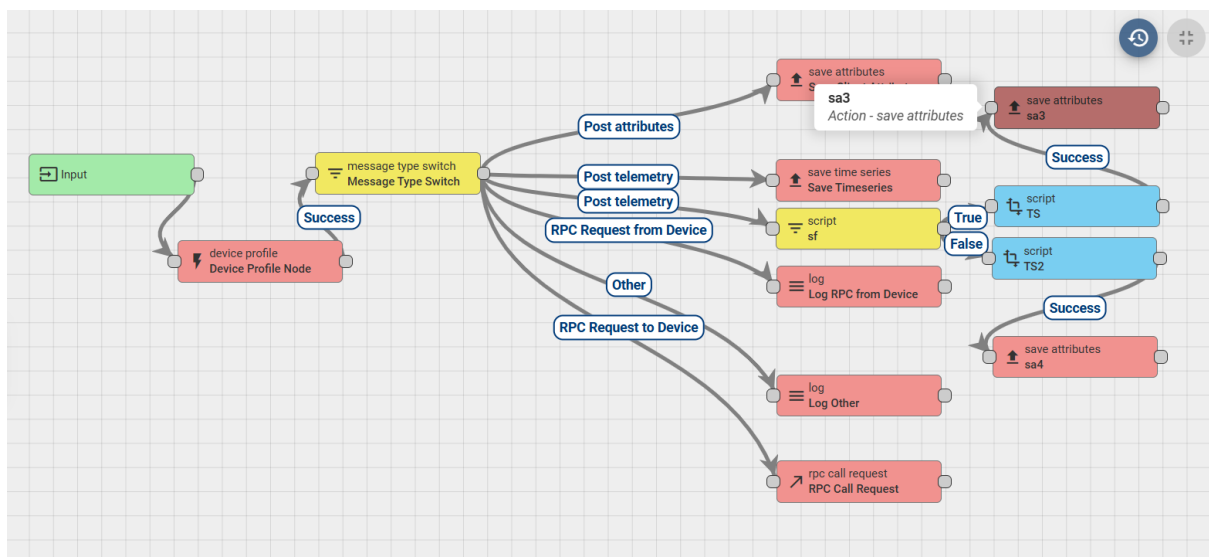
Although the current implementation allows users to manually control the relay through the ThingsBoard dashboard, the system can be further enhanced by utilizing the **ThingsBoard Rule Chain** feature for automatic load protection.

In this approach, the Rule Chain continuously monitors the current telemetry published by the RuggedBoard. If the measured current exceeds a predefined threshold, the Rule Chain can automatically generate a relay control command and transmit it to the embedded device through MQTT. The RuggedBoard receives the command and switches the relay OFF, thereby disconnecting the load without requiring user intervention.

Similarly, when the current returns to a safe operating range, the Rule Chain can be configured to restore the relay state automatically or notify the operator before re-energizing the load, depending on the application requirements.

For example, during testing, a threshold of **400 mA** can be configured. If the measured current exceeds this value, the Rule Chain can automatically trigger the relay to switch OFF, protecting the connected load from excessive current conditions.

The use of Rule Chains enables the system to evolve from a monitoring platform into an intelligent Industrial IoT solution capable of performing automated decision-making and protective actions. Such automation is particularly beneficial in industrial environments where rapid response to abnormal operating conditions is essential.



CHAPTER 9

CONCLUSION

The Industrial Energy Management System developed during this internship successfully demonstrated the integration of embedded hardware, Industrial Internet of Things (IIoT) technologies, and cloud-based monitoring into a practical energy monitoring solution. The project was implemented using the **RBA5D2X Rugged Board**, **ACS712-20A Hall-effect current sensor**, **RS485 relay module**, **Embedded Linux**, **Embedded C**, **MQTT communication protocol**, and **ThingsBoard Community Edition**.

The embedded application continuously acquired current measurements through the Analog-to-Digital Converter (ADC), calculated RMS current, electrical power, and cumulative energy consumption, and published the processed data to the ThingsBoard dashboard using the MQTT protocol. The dashboard provided real-time visualization of current, power, and energy while also enabling remote relay control and energy reset functionality. The implementation of persistent energy storage ensured that accumulated energy values were retained even after restarting the application.

The developed system was successfully validated using **60 W** and **100 W incandescent bulbs** as test loads. Experimental testing confirmed reliable current measurement, accurate power calculation, continuous energy accumulation, stable MQTT communication, successful dashboard visualization, and effective relay operation. The project demonstrated seamless interaction between sensing hardware, embedded software, industrial communication protocols, and cloud services.

Throughout the internship, practical knowledge was gained in embedded Linux application development, ADC interfacing, Industrial IoT communication using MQTT, dashboard development using ThingsBoard, RS485-based relay control, and integration of hardware and software into a complete embedded system. The project also provided valuable experience in system integration, testing, troubleshooting, and technical documentation.

The Industrial Energy Management System developed during this internship provides a scalable foundation for future industrial monitoring applications. With additional features such as automated rule-based control, multi-load monitoring, three-phase energy measurement, and predictive analytics, the system can be further enhanced to meet the requirements of larger industrial environments.

Overall, the objectives of the internship were successfully achieved, and the developed prototype demonstrates a practical, reliable, and cost-effective approach for real-time industrial energy monitoring and remote load management.

